

Module 14: File I/O

Introduction to C++ Programming — Part 2: Intermediate C++

Module Overview

The satisfying, tangible module: students can literally close the terminal, reopen it, run the program again, and see their own data still there. Build the session around that moment rather than the mechanics of ifstream/ofstream in isolation - the mechanics are simple once the motivation is felt.

Suggested Duration: 60–75 minutes

Learning Objectives

By the end of this module, students will be able to:

- Explain why `std::ofstream/std::ifstream` use the same `<< / >>` syntax as `cout/cin`.
- Write data to a file and read it back, checking that the file actually opened.
- Use `std::ios::app` to append instead of overwrite.
- Read a file line by line with `getline`, including splitting a CSV line on a delimiter.
- Convert a text field back into a number with `std::stod/std::stoi`.

Prerequisites

- Module 3 (variables, `std::cin/getline`) from Part 1 - file streams reuse those exact tools.

Materials & Instructor Prep

- A short 'before/after' demo plan: run a program, close the terminal fully, reopen it, run again - staged in advance so the persistence moment lands cleanly live.

Key Vocabulary

Term	Definition
Stream	A general abstraction for reading/writing a sequence of data - <code>std::cout</code> , <code>std::cin</code> , and file streams are all streams.
<code>std::ofstream</code>	"Output file stream" - opens a file for writing.

Term	Definition
<code>std::ifstream</code>	"Input file stream" - opens a file for reading.
<code>std::ios::app</code>	A flag that opens a file in append mode, adding to existing content instead of erasing it.
<code>std::stringstream</code>	Lets you apply stream tools (>>, getline) to a string already in memory, as if it were a file.

Lesson Outline & Instructor Talking Points

1. The motivating moment, staged (10 min)

- Before explaining anything, run a pre-built version of the persistence demo: write a value, close the program, run it again, show the value is still there. Let this land as the 'why' before any syntax appears.

Instructor note: Prepare this demo file in advance, outside class time - don't build it live before explaining the motivation, or the moment gets buried in typing.

2. Writing: `std::ofstream` (15 min)

- Live-code examples/01_writing_to_a_file.cpp. Point out explicitly: `outFile << value` works exactly like `std::cout << value` - same operator, different destination.
- Highlight if `(!outFile)` as a real, not optional, check - ask 'what real-world reasons could this fail?' (folder doesn't exist, no write permission) before explaining.

3. Reading: `std::ifstream` and the `while (stream >> var)` idiom (20 min)

- Live-code examples/02_reading_from_a_file.cpp. Spend real time on `while (inFile >> reading)` - ask 'how does this loop know when to stop, without us telling it how many values are in the file?' before explaining that the stream itself evaluates false at end-of-file.
- Connect explicitly to Module 15 (next module): `std::stoi`, which we'll use to convert file text into numbers, uses this exact same signal-something-went-wrong idea, formalized as an exception.'

Instructor note: The `stream-evaluates-to-false` idiom is genuinely non-obvious the first time. Don't rush past it even though the code itself is short.

4. Appending and `getline` (15 min)

- Run examples/03_appending_and_getline.cpp twice in a row, showing the log file grows by one line each run - this is the concrete proof that `std::ios::app` works as described.

5. A simple CSV pattern (15 min)

- Run examples/04_csv_with_stringstream.cpp. Frame `std::stringstream` honestly as 'a trick worth recognizing, not a tool you need to master deeply for this course' - the pattern (wrap a line in a `stringstream`, `getline` with a `' '` delimiter) is what matters.

Common Student Misconceptions

Misconception	How to Address It
"If I open a file with <code>ofstream</code> , my new data gets added to what's already there."	Default <code>ofstream</code> behavior OVERWRITES the file's previous contents - <code>std::ios::app</code> is required to append instead.
"A file that fails to open still lets the program read/write normally, just with no data."	Reads/writes on a failed stream silently do nothing useful - always check if <code>(!file)</code> immediately after opening, before using it.
"Every field read from a file is already the right type to do math with."	Everything read with <code>getline</code> is text (<code>std::string</code>), even a field that looks numeric - <code>std::stod/std::stoi</code> convert it explicitly.

Exercises & Module Project

Exercise: 14-file-io/exercises/exercise1.cpp - an appending CSV log with a running average, run multiple times to see accumulation. Project (three tracks): Generic - Expense Log; EE - Sensor Data Logger; Biomedical - Patient Vitals Log. All three build a `std::ios::app` CSV logger read back and summarized on every run.

Facilitation guidance: Require students to actually run their program 2-3 times during the demo/review, not just once - the entire pedagogical point of this project is watching data persist ACROSS runs, which a single execution can't demonstrate.

Discussion & Comprehension Check Questions

1. Why does `while (inFile >> reading)` know when to stop reading, with no explicit count of values needed?

Guidance: The stream itself evaluates to false once it hits the end of the file (or a read error) - the loop condition IS that check, each iteration.

2. What real-world reasons could cause `std::ofstream` to fail to open a file?

Guidance: The target folder doesn't exist, the program lacks write permission, the disk is full, or the path is invalid - always worth checking `if (!file)` rather than assuming success.

Assessment Ideas

- Have students predict, in writing, the exact contents of a small file after a sequence of `ofstream/ios::app` operations described in pseudocode - before running any real code.

Answer Key Notes

See `14-file-io/exercises/solutions/exercise1_solution.cpp`.

Wrap-Up & Bridge to Next Module

Bridge forward: "Your file-reading code right now trusts every field is valid. Next module gives you a structured way to handle the case where it isn't - exception handling."

Instructor Tips

- Have students delete their generated `.csv/.txt` test files before committing, or mention the repo's `.gitignore` already excludes them - avoid a class full of confusing merge/diff noise from personal test data.